BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 1

INTRODUCTION TO WEB PROGRAMMING

Unit 5

HTML Forms

Table of Contents

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	3
	1.1 Learning Objectives	-	-	
2	Introduction to HTML Forms	-	-	4-5
3	Form Elements	-	-	6-10
4	Input Types	-	-	10-15
5	Form Attributes	-	-	15-19
6	Summary	-	-	19
7	Questions	-	1	20-21
8	Answers	-	-	21-22

1. INTRODUCTION

An HTML `<form>` element serves as a vital tool for gathering user input and includes a variety of interactive elements. Within a form, you can incorporate input fields for text, numbers, email addresses, passwords, as well as control elements such as checkboxes, radio buttons, and submit buttons. In essence, a form acts as a container housing these input elements, allowing users to provide essential information. For instance, when a user wishes to purchase a product online, they must first fill out an address form with their shipping details and then proceed to the payment form to input their payment information, culminating in the completion of their order.

Designing forms in HTML involves strategically placing these input fields within paragraphs, preformatted text, lists, or tables. This strategic placement offers significant flexibility in shaping the visual layout of the forms, allowing web developers to create forms that not only serve their functional purpose but also harmonize seamlessly with the overall design and usability of the website. In sum, HTML forms play a pivotal role in user interaction and data collection on the web, making them a fundamental component for various online processes, including user registration, e-commerce transactions, surveys, and more. Their versatile design options ensure that they can be integrated effectively into websites while maintaining a structured and user-friendly data input experience.

1.1 Learning Objectives

By the end of this chapter you will be able to :

- *Identify the basic components of an HTML form, such as the <form> element and form controls.*
- *Differentiate between block-level and inline-level form elements.*
- *Create a functional HTML form that incorporates various input types for data entry.*
- *Evaluate how form attributes impact the behavior and functionality of web forms.*
- *Evaluate the user experience and security implications of different form submission methods (e.g., POST and GET).*

2. INTRODUCTION TO HTML FORMS

HTML forms are a crucial component of web development that allow users to input data, interact with websites, and submit information to web servers for processing. HTML forms are created using the `<form>` element and can include various types of input fields, buttons, and other form controls. Here's an introduction of HTML forms and their key components:

1. **`<form>` Element:**
   - The `<form>` element is used to create an HTML form. It serves as a container for form controls and defines the action to be taken when the form is submitted. The `action` attribute specifies the URL where the form data should be sent for processing, typically a server-side script or endpoint.
2. **Form Controls:**
   - Form controls are the input fields and widgets that users interact with to provide data. Common form controls include:
   - Text input fields (`<input type="text">`) for single-line text input.
   - Textarea fields (`<textarea>`) for multi-line text input.
   - Radio buttons (`<input type="radio">`) and checkboxes (`<input type="checkbox">`) for selecting options.
   - Dropdown lists (`<select>`) for selecting options from a list.
   - Buttons (`<input type="submit">`, `<input type="reset">`, and `<button>`) for form submission, clearing, and custom actions.
3. **`<label>` Element:**
   - The `<label>` element is used to provide a text label for form controls, enhancing accessibility and user experience. It is associated with a specific input field using the `for` attribute and the input's `id` attribute.
4. **`name` and `id` Attributes:**
   - Each form control should have a `name` attribute, which serves as the identifier for the input field when the form is submitted. Additionally, the `id` attribute can be used to uniquely identify form controls and associate them with `<label>` elements.
5. **Form Submission:**
   - When a user interacts with form controls and submits the form, the browser collects the data from the input fields and sends it to the URL specified in the `action`

attribute of the `<form>` element using the HTTP POST or GET method, as determined by the `method` attribute (commonly POST).

- Server-side scripts or applications receive this data and can process it accordingly.

6. **Validation and Error Handling:**
   - HTML forms can include client-side and server-side validation to ensure data accuracy and completeness. Client-side validation is performed using JavaScript to provide instant feedback to users, while server-side validation is a crucial security measure to validate data on the server.
7. **Styling and Layout:**
   - CSS can be used to style and format the appearance of form controls, making them visually appealing and consistent with the overall design of the website.

In summary, HTML forms are a fundamental part of web development, enabling user interaction and data submission. They consist of the `<form>` element as a container, various form controls for data input, and optional labels and validation mechanisms. When a user submits a form, the data is sent to a server for processing, making forms essential for a wide range of web applications, including user registration, login, search, and data collection.

3. FORM ELEMENTS

The `<form>` element in HTML is used to create interactive forms that allow users to input data and submit it to a server for processing. It serves as a container for various form controls, such as text input fields, checkboxes, radio buttons, and buttons. The `action` attribute of the `<form>` element specifies the URL where the form data should be sent when the user submits the form. Additionally, the `method` attribute defines the HTTP method (usually GET or POST) used to send the data to the server. Here's a detailed explanation of the `<form>` element with syntax and examples:

The HTML <form> element can contain one or more of the following form elements:

- <input>
- <label>
- <select>
- <textarea>
- <button>
- <fieldset>
- <legend>
- <datalist>
- <output>
- <option>
- <optgroup>

Let's explore each of the HTML form elements in detail:

1. **`<input>` Element:**
   - The `<input>` element is one of the most versatile form elements. It allows users to enter data or make selections using various input types, such as text, password, radio buttons, checkboxes, and more. The `type` attribute specifies the input type.
2. **`<label>` Element:**
   - The `<label>` element is used to provide a textual description or label for form controls, making them more accessible and user-friendly. It is often associated with an `<input>` element using the `for` attribute and the input's `id`.

3. **`<select>` Element:**
   - The `<select>` element creates a dropdown list or selection box. It contains one or more `<option>` elements, and users can choose one or more options from the list. It's commonly used for selecting items from a list of choices.
4. **`<textarea>` Element:**
   - The `<textarea>` element provides a multiline text input field, allowing users to enter larger blocks of text or comments. It's useful for fields where longer textual input is required.
5. **`<button>` Element:**
   - The `<button>` element creates a clickable button that can trigger form submission or perform other actions when clicked. It can be used for actions like submitting a form, resetting form fields, or triggering JavaScript functions.
6. **`<fieldset>` Element:**
   - The `<fieldset>` element groups related form elements together and provides a visual and structural container for them. It's often used to group radio buttons or checkboxes that belong to the same category.
7. **`<legend>` Element:**
   - The `<legend>` element is used within a `<fieldset>` element and provides a title or caption for the group of form controls enclosed within that `<fieldset>`. It describes the purpose of the group.
8. **`<datalist>` Element:**
   - The `<datalist>` element is used in conjunction with an `<input>` element of type "text" or "number." It provides a list of predefined options that can be displayed as suggestions while the user types in the input field.
9. **`<output>` Element:**
   - The `<output>` element is used to display the result or output of a calculation or operation performed using JavaScript. It's often used for real-time feedback to users.
10. **`<option>` Element:**
    - The `<option>` element is used within a `<select>` element to define individual options within the dropdown list. Each `<option>` represents a choice that the user can select.

11. `<optgroup>` Element:

- The `<optgroup>` element groups related `<option>` elements within a `<select>` dropdown. It provides a way to categorize and organize the available choices.

These form elements play a crucial role in creating interactive and user-friendly web forms, allowing websites to collect various types of data from users and provide a structured and accessible user interface for data entry and interaction.

Syntax:

```
<form action="processing-script-url" method="GET/POST">
  <!-- Form controls go here -->
</form>
```

Explanation:

- `<form>`: This is the opening tag of the form element.
- `action`: The `action` attribute specifies the URL where the form data will be sent for processing. It can be an absolute or relative URL, typically pointing to a server-side script that handles the form data.
- `method`: The `method` attribute specifies the HTTP method to be used when submitting the form. The two common methods are:
- `GET`: Appends the form data to the URL as query parameters. Suitable for search queries and forms with limited data.
- `POST`: Sends the form data in the request body. Suitable for forms with sensitive or large data.

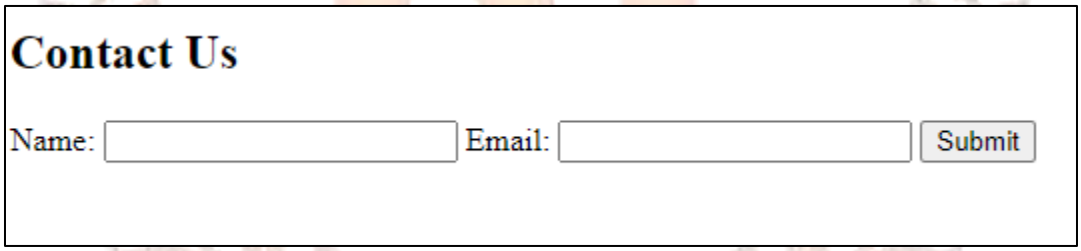
Example:

Let's create a simple HTML form that collects a user's name and email address and sends the data to a server for processing using the POST method.

```
<!DOCTYPE html>
<html>
<head>
  <title>Sample Form</title>
</head>
```

```
<body>
  <h2>Contact Us</h2>
  <form action="process.php" method="POST">
    <label for="name">Name:</label>
    <input type="text" id="name" name="name" required>
        <label for="email">Email:</label>
    <input type="email" id="email" name="email" required>
        <button type="submit">Submit</button>
  </form>
</body>
</html>
```

The output of the above code will be :



In this example:

- The `<form>` element serves as the container for the form controls.
- The `action` attribute is set to "process.php," which is the URL where the form data will be sent for processing.
- The `method` attribute is set to "POST" to securely send the data in the request body.
- Two form controls, `<input>` elements for name and email, are included with associated `<label>` elements.
- The `for` attribute of `<label>` elements is used to associate them with the corresponding `<input>` elements using their `id` attributes.
- The "required" attribute is added to both input elements to indicate that they must be filled out before the form can be submitted.
- A submit button is provided to allow the user to submit the form.

When the user submits this form, the data entered in the input fields (name and email) will be sent to the "process.php" script on the server for further processing. The method and action attributes of the form play a crucial role in determining how this data is sent and where it goes.

4. FORM INPUT TYPES

HTML **<input>** elements are incredibly versatile and can be used to create various types of form controls for user input. The attribute of the **<input>** element specifies the type of input control to be displayed. Here are some common **<input>** types :

1. <input type="button">
2. <input type="checkbox">
3. <input type="color">
4. <input type="date">
5. <input type="datetime-local">
6. <input type="email">
7. <input type="file">
8. <input type="hidden">
9. <input type="image">
10. <input type="month">
11. <input type="number">
12. <input type="password">
13. <input type="radio">
14. <input type="range">
15. <input type="reset">
16. <input type="search">
17. <input type="submit">
18. <input type="tel">
19. <input type="text">
20. <input type="time">
21. <input type="url">
22. <input type="week">

Let’s explore each input type in detail :

Certainly, here's a more detailed explanation of each HTML `<input>` type with examples:

1. `type="button"`:
   - Creates a clickable button that can trigger JavaScript functions.

```
<input type="button" value="Click me" onclick="alert('Button clicked!');">
```

2. `type="checkbox"`:
   - Renders a checkbox that allows users to select or deselect an option.

```
<input type="checkbox" id="subscribe" name="subscription" value="yes">
```

3. `type="color"`:
   - Provides a color picker for selecting a color.

```
<input type="color" id="colorPicker" name="color">
```

4. `type="date"`:
   - Displays a date picker for selecting a date.

```
<input type="date" id="birthdate" name="birthdate">
```

5. `type="datetime-local"`:
   - Allows users to input both date and time values.

```
<input type="datetime-local" id="datetime" name="datetime">
```

6. `type="email"`:
   - Accepts email addresses and includes built-in validation.

```
<input type="email" id="userEmail" name="email" placeholder="Enter your email" required>
```

7. `type="file"`:
   - Creates a file upload control for uploading files.

```
<input type="file" id="fileUpload" name="file">
```

8. `type="hidden"`:
   - Hides the input field from the user but can be used to store data on the server.

```
<input type="hidden" name="token" value="abc123">
```

9. `type="image"`:
   - Functions like a clickable image that can trigger JavaScript actions.

```
<input type="image" src="submit.png" alt="Submit" onclick="submitForm()">
```

10. `type="month"`:
   - Allows users to select a specific month and year.

```
<input type="month" id="monthPicker" name="month">
```

11. `type="number"`:
   - Accepts numeric input, including integers and floating-point numbers.

```
<input type="number" id="quantity" name="quantity" min="1" max="100" step="1">
```

12. `type="password"`:
   - Hides the entered text (commonly used for password fields).

```
<input type="password" id="password" name="password" placeholder="Enter your password" required>
```

13. `type="radio"`:
   - Provides radio buttons for exclusive selection from a group.

```
<input type="radio" id="male" name="gender" value="male">

<label for="male">Male</label>
```

14. `type="range"`:
   - Creates a slider control for selecting a numeric value within a specified range.

```
<input type="range" id="slider" name="range" min="0" max="100" step="1">
```

15. `type="reset"`:
   - Resets all form controls to their default values.

```
<input type="reset" value="Reset Form">
```

16. `type="search"`:
   - Optimized for search queries with built-in search functionalities.

```
<input type="search" id="searchQuery" name="search" placeholder="Search...">
```

17. `type="submit"`:
    - Submits the form data to the server for processing.

```
<input type="submit" value="Submit Form">
```

18. `type="tel"`:
    - Used for telephone numbers and can include input masks for specific formats.

```
<input type="tel" id="phoneNumber" name="phone" placeholder="Enter your phone number">
```

19. `type="text"`:
    - Basic text input field for alphanumeric characters and plain text.

```
<input type="text" id="username" name="username" placeholder="Enter your name" required>
```

20. `type="time"`:
    - Allows users to select a specific time.

```
<input type="time" id="appointmentTime" name="time">
```

21. `type="url"`:
    - Designed for website URLs and includes URL validation.

```
<input type="url" id="websiteUrl" name="url" placeholder="Enter a URL">
```

22. `type="week"`:
    - Permits users to choose a specific week and year.

```
<input type="week" id="weekSelector" name="week">
```

These HTML input types cater to a wide range of data input requirements and enhance user interaction on web forms. They can be customized with attributes to enforce validation, set value ranges, and more, depending on the specific needs of your application.

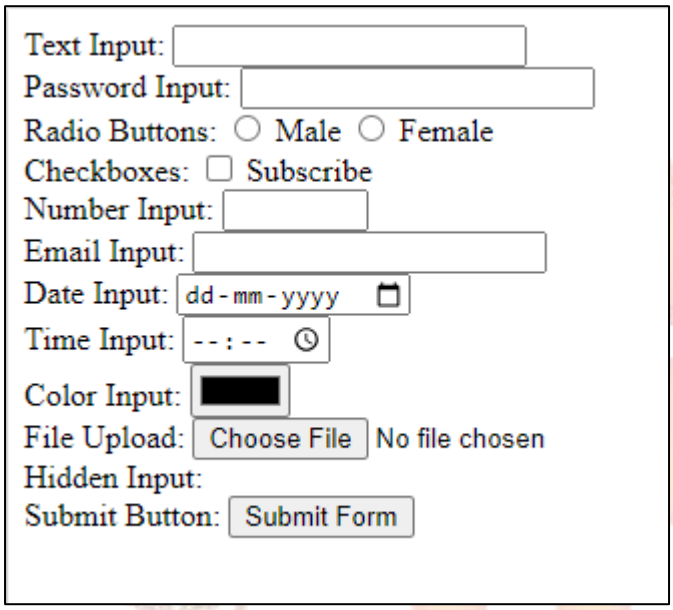
Here is an example program depicting all the input types stated above :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>All Input Types</title></head><body>
<form action="#" method="post">
  <label for="text">Text Input:</label>
  <input type="text" id="text" name="text"><br>
  <label for="password">Password Input:</label>
  <input type="password" id="password" name="password"><br>
  <label>Radio Buttons:</label>
  <input type="radio" id="male" name="gender" value="male">
  <label for="male">Male</label>
  <input type="radio" id="female" name="gender" value="female">
  <label for="female">Female</label><br>
  <label>Checkboxes:</label>
  <input type="checkbox" id="subscribe" name="subscription" value="yes">
  <label for="subscribe">Subscribe</label><br>
  <label for="number">Number Input:</label>
  <input type="number" id="number" name="number" min="1" max="100"
step="1"><br>
  <label for="email">Email Input:</label>
  <input type="email" id="email" name="email"><br>
  <label for="date">Date Input:</label>
  <input type="date" id="date" name="date"><br>
  <label for="time">Time Input:</label>
  <input type="time" id="time" name="time"><br>
  <label for="color">Color Input:</label>
  <input type="color" id="color" name="color"><br>
  <label for="file">File Upload:</label>
  <input type="file" id="file" name="file"><br>
  <label for="hidden">Hidden Input:</label>
  <input type="hidden" id="hidden" name="hidden" value="hidden-value"><br>
  <label for="submit">Submit Button:</label>
  <input type="submit" id="submit" value="Submit Form"><br>
```

```
</form></body></html>
```

The output will be given as :



5. FORM ATTRIBUTES

HTML `<form>` elements can have several attributes that control their behavior and appearance. Here are some of the most commonly used form attributes explained in detail:

1. `action` Attribute:
   - The `action` attribute specifies the URL where the form data should be sent when the form is submitted. This URL typically points to a server-side script that processes the form data.

```
<form action="submit.php" method="post">
```

2. `method` Attribute:
   - The `method` attribute defines how the form data should be sent to the server. It can be either `GET` or `POST`.
   - `GET` appends the form data to the URL, making it visible in the address bar. It's suitable for search queries and when data submission doesn't alter server data.
   - `POST` sends the data in the body of the HTTP request, which is more secure and suitable for sensitive data.

```
<form action="submit.php" method="post">
```

3. `name` Attribute:
   - The `name` attribute specifies a name for the form, which is used to identify the form when submitting multiple forms on a page.

```
<form name="registrationForm" action="submit.php" method="post">
```

4. `target` Attribute:
   - The `target` attribute specifies where the response from the server should be displayed. It's often used with frames or iframes. Common values are `_blank` (new tab or window) and `_self` (current frame or window).

```
<form action="submit.php" method="post" target="_blank">
```

5. `enctype` Attribute:
   - The `enctype` attribute determines how the form data should be encoded before sending it to the server. Common values include:
   - `application/x-www-form-urlencoded` (default): Form data is URL-encoded, suitable for most text data.
   - `multipart/form-data`: Used for file uploads, it encodes binary data.
   - `text/plain`: The form data is sent as plain text.

```
<form action="upload.php" method="post" enctype="multipart/form-data">
```

6. `autocomplete` Attribute:
   - The `autocomplete` attribute controls whether the browser should automatically complete form fields based on user input history. Values can be `on` (default), `off`, or specific input types like `username`, `current-password`, etc.

```
<form action="login.php" method="post" autocomplete="on">
```

7. `novalidate` Attribute:
   - The `novalidate` attribute, when present, disables the browser's built-in form validation. This is useful when custom JavaScript validation is applied.

```
<form action="submit.php" method="post" novalidate>
```

8. `onsubmit` Attribute:

- The `onsubmit` attribute specifies a JavaScript function to be executed when the form is submitted. It can be used for custom form validation or to perform other actions before submission.

```
<form action="submit.php" method="post" onsubmit="return validateForm()">
```

These are some of the essential attributes you can use with HTML `<form>` elements. Depending on your specific use case, you may also use additional attributes and JavaScript to enhance the functionality and behavior of your forms.

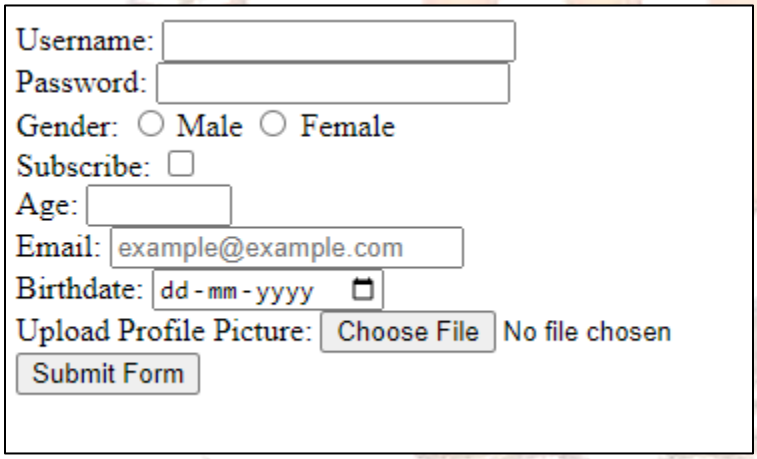
Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Form Attributes Example</title>
</head>
<body>
  <form action="submit.php" method="post" name="registrationForm" target="_blank" enctype="multipart/form-data" autocomplete="on">
    <label for="username">Username:</label>
    <input type="text" id="username" name="username" required><br>
    <label for="password">Password:</label>
    <input type="password" id="password" name="password" required><br>
    <label>Gender:</label>
    <input type="radio" id="male" name="gender" value="male">
    <label for="male">Male</label>
    <input type="radio" id="female" name="gender" value="female">
    <label for="female">Female</label><br>
    <label for="subscription">Subscribe:</label>
    <input type="checkbox" id="subscription" name="subscription" value="yes"><br>

    <label for="number">Age:</label>
```

```
    <input type="number" id="age" name="age" min="18" max="100" required><br>
    <label for="email">Email:</label>
    <inputtype="email" id="email" name="email"
placeholder="example@example.com"><br>
    <label for="birthdate">Birthdate:</label>
    <input type="date" id="birthdate" name="birthdate"><br>
    <label for="file">Upload Profile Picture:</label>
    <input type="file" id="profilePicture" name="profile_picture"><br>
    <input type="hidden" name="form_type" value="registration">
        <input type="submit" value="Submit Form">
  </form></body></html>
```

The output will be given as:



In this example:

- The form has an action attribute set to "submit.php," which is where the form data will be sent.
- The method attribute is set to "post" to send data in the HTTP request body.
- The form has a name attribute, "registrationForm," to identify it.
- The target attribute is set to "_blank," which will open the form submission response in a new tab or window.
- The enctype attribute is set to "multipart/form-data" to enable file uploads.
- The autocomplete attribute is set to "on" to allow the browser to suggest autocompletions.

- There's a combination of text, password, radio buttons, checkboxes, number, email, date, and file input fields.
- A hidden input field is included with the name "form_type" and value "registration."
- The submit button triggers the form submission.

This example demonstrates how various form attributes can be used to control the behavior and appearance of an HTML form.

6. SUMMARY

The chapter serves as a comprehensive introduction to HTML forms and their integral components. It elucidates the significance of HTML forms in web development, elucidating their pivotal role in collecting user input and enabling interactive web applications. The chapter then delves into the core building blocks of forms, elucidating the purpose and usage of form elements like input fields, labels, select boxes, and buttons in creating user-friendly and engaging forms. It further explores the diverse input types available, ranging from text and passwords to radio buttons and file uploads, providing insight into how each input type is employed to gather specific user data. Lastly, the chapter explains the essential form attributes that govern form behavior and appearance, emphasizing attributes such as `action`, `method`, `name`, `target`, `enctype`, `autocomplete`, and `novalidate`. This comprehensive understanding equips web developers with the necessary knowledge to design and implement interactive web forms efficiently, enhancing user engagement and data collection capabilities.

7 QUESTIONS

Self-Assessment Questions

SELF-ASSESSMENT QUESTIONS – 1

1. What is the primary purpose of HTML forms in web development?
2. Name one essential component of HTML forms that is used for creating interactive form fields.
3. Which HTML element is used to define a label for an input field?
4. What is the significance of the <select> element in HTML forms?
5. Which input type is typically used for collecting email addresses from users?
6. How can you create a radio button input in HTML?
7. What does the action attribute in an HTML form specify?
8. Which input type is suitable for uploading files?
9. What does the novalidate attribute do in an HTML form?
10. How can you open the form submission response in a new tab or window?

Terminal Questions

1. Explain the main purpose of HTML forms in web development. Provide an example of a real-world scenario where forms are commonly used and describe how they enhance user interaction.
2. Describe the key components of an HTML form, including form elements, and their roles in creating user-friendly and interactive forms. Provide examples of different form elements and their usage.
3. Explore the various input types available in HTML forms, such as text, email, checkbox, and date. For each input type, explain its purpose, and provide an example of when it would be appropriate to use it in a web form.
4. Discuss the significance of form attributes in HTML. Choose two form attributes (e.g., action and method) and explain in detail how they impact the behavior and functionality of a form. Provide examples to illustrate your explanations.

5. When designing an HTML form, it's important to consider user experience and accessibility. Explain how the use of labels and other accessibility features can enhance the usability of a web form. Provide an example of a form with proper labeling and accessibility attributes.
6. HTML forms often require validation to ensure that user-submitted data meets certain criteria. Describe the methods and techniques for implementing form validation in HTML forms. Provide an example of a form with client-side validation and explain how it works.

8. ANSWERS

Self-Assessment Questions

1. The primary purpose of HTML forms in web development is to collect and process user input.
2. One essential component of HTML forms used for creating interactive form fields is the <input> element.
3. The <label> element is used to define a label for an input field in HTML forms.
4. The <select> element is used to create a dropdown list or select box in HTML forms.
5. The <input type="email"> input type is typically used for collecting email addresses from users.
6. Radio buttons in HTML forms are created using the <input type="radio"> element.
7. The action attribute in an HTML form specifies the URL where the form data should be sent for processing, usually a server-side script or endpoint.
8. The <input type="file"> input type is suitable for uploading files in HTML forms.
9. The novalidate attribute in an HTML form disables the browser's built-in form validation.
10. To open the form submission response in a new tab or window, you can use the target="_blank" attribute in the form tag like this: <form action="submit.php" method="post" target="_blank">.

Terminal Questions

1. Refer section 5.3
2. Refer section 5.3
3. Refer section 5.4
4. Refer section 5.6
5. Refer section 5.5
6. Refer section 5.4

INSPIRED BY LIFE